Shri Ramdeobaba College of Engineering & Management, Nagpur

Course: Machine Learning **Course Code: CDT5002**
Solution set: Test 1 **Date: 09/09/2025**

Q.1 a) Consider the following set of training examples and construct a Decision Tree using ID3 algorithm.

Instance	a_1	a_2	Classification
1	T	T	+
2	T	T	+
3	T	F	-
4	F	F	+
5	F	T	-
6	F	T	-

(a) What is the entropy of this collection of training examples with respect to the target function classification?

(b) What is the information gain of a_1 and a_2 relative to these training examples? (7 Marks)

Solution:

Step 1: Calculate the Entropy of the Entire Dataset

$$\text{Entropy(S)} = -\sum_{i=1}^{c} p_i log_2 p_i$$

$p_{yes} = 3/6 = 0.5$
$p_{no} = 3/6 = 0.5$

$$\text{Entropy(S)} = -\left(\frac{3}{6} log_2 \frac{3}{6} + \frac{3}{6} log_2 \frac{3}{6}\right) = 1$$

Step 2: Calculate the Entropy for a_2 Feature

For ` $a_2$`:

T: 4 samples (2 +, 2 -)
F: 2 samples (1 +, 1 -)

Entropy for each subset:

$$\text{Entropy(T)} = -\left(\frac{2}{4} log_2 \frac{2}{4} + \frac{2}{4} log_2 \frac{2}{4}\right) = 1$$

$$\text{Entropy(F)} = -\left(\frac{1}{2} log_2 \frac{1}{2} + \frac{1}{2} log_2 \frac{1}{2}\right) = 1$$

Weighted Entropy for "a_2":

$$\text{Weighted Entropy} = \left(\frac{4}{6} \times 1 + \frac{2}{6} \times 1\right) = 1$$

Information Gain for "a_2":
Information Gain (S, a_2) = Entropy (S) − Weighted Entropy = 1 − 1 = 0

Calculate the Entropy for "a_1" Feature

For ` $a_1$`:

T: 3 samples (2 +, 1 -)
F: 3 samples (1 +, 2 -)

Entropy for each subset:

$$\text{Entropy(T)} = -\left(\frac{2}{3}log_2\frac{2}{3} + \frac{1}{3}log_2\frac{1}{3}\right) = 0.91829$$

$$\text{Entropy(F)} = -\left(\frac{1}{3}log_2\frac{1}{3} + \frac{2}{3}log_2\frac{2}{3}\right) = 0.91829$$

Weighted Entropy for "a_1":

$$\text{Weighted Entropy} = \left(\frac{3}{6} \times 0.91829 + \frac{3}{6} \times 0.91829\right) = 0.91829$$

Information Gain for "a_1":
Information Gain (S, a_1) = Entropy (S) − Weighted Entropy = 1 – 0.91829 = 0.08171

Step 3: Choose the Best Feature to Split- with highest IG
"a_1": 0.08171

For the next Split, compare the individual feature values of "a_1" with the other features "a_2":
Starting with:

1) $a_1 - T$

Instance	a_1	a_2	Classification
1	T	T	+
2	T	T	+
3	T	F	-

Samples = 3, + : 2, -ve : 1

For T & a_1 calculate the Entropy:
T: 2 Samples: 2 : +, 0 : -ve, Entropy = 0
F: 1 Sample: 0 : +, 1 : -ve, Entropy = 0

Solving further, we will form the ID3 Decision tree as show below:

```
                [a1]
               /    \
           T  /      \  F
             /        \
        [a2]          [a2]
       /    \        /    \
  a2=T/      \a2=F  a2=F/    \a2=T
    +         -      +       -
```

Decision rules (readable)

1. If `a1 = T` and `a2 = T` → +
2. If `a1 = T` and `a2 = F` → −
3. If `a1 = F` and `a2 = F` → +
4. If `a1 = F` and `a2 = T` → −

Q.2 a) Write k-Nearest Neighbor regression algorithm for Regression and apply the k-NN regression algorithm to predict the value of "Y" for a new query point (X1, X2) = (2.5, 3.5). Take value of k = 3 and 5.

Instance No.	X1	X2	Target function(Y)
1	1.2	2.3	1.8
2	3.4	1.5	3.2
3	2.2	3.1	2.9
4	1.9	2.8	2.3
5	3.1	4.0	3.5
6	2.0	3.5	2.7
7	1.5	1.0	1.4
8	3.7	3.2	3.8

(5 Marks)

Solution:

k-Nearest Neighbors Regression Algorithm

The k-Nearest Neighbors (k-NN) algorithm for regression is a non-parametric method that predicts the target value for a new query point by averaging the target values of its k closest training examples in the feature space. It assumes that similar inputs (based on distance) have similar outputs. The algorithm proceeds as follows:

Training algorithm: [As per Tom Mitchell]

For each training example <*x*, f *(x)*> , add the example to the list *training_examples*

Classification algorithm:

- Given a query instance *xq* to be classified,
- Let xl . . .*xk* denote the *k* instances from *training examples* that are nearest to *xq*

to approximate a real-valued target function,

Return

$$f : \mathfrak{R}^d \rightarrow \mathfrak{R} \qquad \hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} f(x_i)}{k}$$

Numerical solution:

Step 1: Compute Distances of all instances from test point, (X1, X2) = (2.5, 3.5) using Euclidian distance formula.

Instance No.	Distance from Test Point
1	1.769
2	2.193
3	0.5
4	0.922
5	0.781
6	0.5
7	2.693
8	1.237

Step 2: Sort them in ascending order.

When k=3: 3- 0.5, 6- 0.5, 5- 0.781

When k=5: 3- 0.5, 6- 0.5, 5- 0.781, 4- 0.922, 8- 1.237

Step 3: Average values:

For k=3: (2.9 + 2.7 + 3.5)/3 = 3.0333

For k=5: (2.9 + 2.7 + 3.5 + 2.3 + 3.8)/5 = 3.04

Q.2 b) True or False? Write answers with proper justification:

i) Increasing the value of k generally leads to lower variance but higher bias.

ii) Lazy Learner builds their learning model during Training. **(2 marks)**

Solution:

i) True **Lower Variance:** When *k* is increased, the model averages or considers more neighbors to make a prediction. This smooths out the decision boundary (in

		classification) or the regression surface (in regression), making the model less sensitive to small fluctuations or noise in the training data. As a result, the variance of the model decreases because predictions are more stable across different training sets. **Higher Bias:** A larger *k* means the model relies on a broader set of neighbors, which can oversimplify the decision boundary or regression function. This makes the model less flexible and more likely to underfit the data, as it may fail to capture complex patterns or local variations in the data. This increases the bias, as the model assumes a simpler structure than the true underlying distribution. ii) False **Lazy Learning:** In lazy learning, the algorithm stores the training data without performing significant computation or constructing a generalized model during the training phase. For example, in k-NN, the training phase involves simply memorizing the training dataset (i.e., storing the feature vectors and their corresponding labels). **Model Building at Prediction Time:** When a prediction is required for a new data point, the lazy learner (e.g., k-NN) computes distances to all training points, identifies the *k* nearest neighbors, and makes a prediction based on those neighbors (e.g., by majority voting for classification or averaging for regression). This process effectively "builds" the decision rule at prediction time, not during training.
Q.3	a)	Derive an expression for weight updation rule in perceptron using gradient descent optimization algorithm. Show step by step derivation. (4 Marks) How can we calculate the direction of steepest descent along the error surface? This direction can be found by computing the derivative of E with respect to each component of the vector $\vec{w}$. This vector derivative is called the *gradient* of E with respect to $\vec{w}$, written $\nabla E(\vec{w})$. $$\nabla E(\vec{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots, \frac{\partial E}{\partial w_n}\right] \quad (4.3)$$ Notice $\nabla E(\vec{w})$ is itself a vector, whose components are the partial derivatives of E with respect to each of the w_i. *When interpreted as a vector in weight space, the gradient specifies the direction that produces the steepest increase in E.* The negative of this vector therefore gives the direction of steepest decrease. For example, the arrow in Figure 4.4 shows the negated gradient $-\nabla E(\vec{w})$ for a particular point in the w_0, w_1 plane. Since the gradient specifies the direction of steepest increase of E, the training rule for gradient descent is $$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$ where $$\Delta\vec{w} = -\eta\nabla E(\vec{w}) \quad (4.4)$$ Here η is a positive constant called the learning rate, which determines the step size in the gradient descent search. The negative sign is present because we want to move the weight vector in the direction that *decreases* E. This training rule can also be written in its component form $$w_i \leftarrow w_i + \Delta w_i$$

How can we calculate the direction of steepest descent along the error surface? This direction can be found by computing the derivative of E with respect to each component of the vector $\vec{w}$. This vector derivative is called the *gradient* of E with respect to $\vec{w}$, written $\nabla E(\vec{w})$.

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots, \frac{\partial E}{\partial w_n}\right] \tag{4.3}$$

Notice $\nabla E(\vec{w})$ is itself a vector, whose components are the partial derivatives of E with respect to each of the w_i. *When interpreted as a vector in weight space, the gradient specifies the direction that produces the steepest increase in* E. The negative of this vector therefore gives the direction of steepest decrease. For example, the arrow in Figure 4.4 shows the negated gradient $-\nabla E(\vec{w})$ for a particular point in the w_0, w_1 plane.

Since the gradient specifies the direction of steepest increase of E, the training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

where

$$\Delta\vec{w} = -\eta\nabla E(\vec{w}) \tag{4.4}$$

Here η is a positive constant called the learning rate, which determines the step size in the gradient descent search. The negative sign is present because we want to move the weight vector in the direction that *decreases* E. This training rule can also be written in its component form

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = -\eta\frac{\partial E}{\partial w_i} \tag{4.5}$$

which makes it clear that steepest descent is achieved by altering each component w_i of $\vec{w}$ in proportion to $\frac{\partial E}{\partial w_i}$.

To construct a practical algorithm for iteratively updating weights according to Equation (4.5), we need an efficient way of calculating the gradient at each step. Fortunately, this is not difficult. The vector of $\frac{\partial E}{\partial w_i}$ derivatives that form the

gradient can be obtained by differentiating E from Equation (4.2), as

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_{d\in D}(t_d - o_d)^2 \\ &= \frac{1}{2}\sum_{d\in D}\frac{\partial}{\partial w_i}(t_d - o_d)^2 \\ &= \frac{1}{2}\sum_{d\in D}2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\ &= \sum_{d\in D}(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w}\cdot\vec{x_d}) \\ \frac{\partial E}{\partial w_i} &= \sum_{d\in D}(t_d - o_d)(-x_{id}) \qquad (4.6)\end{aligned}$$

where x_{id} denotes the single input component x_i for training example d. We now have an equation that gives $\frac{\partial E}{\partial w_i}$ in terms of the linear unit inputs x_{id}, outputs O_d, and target values t_d associated with the training examples. Substituting Equation (4.6) into Equation (4.5) yields the weight update rule for gradient descent

$$\Delta w_i = \eta \sum_{d\in D}(t_d - o_d)\ x_{id} \qquad (4.7)$$

To summarize, the gradient descent algorithm for training linear units is as follows: Pick an initial random weight vector. Apply the linear unit to all training examples, then compute Δw_i for each weight according to Equation (4.7). Update each weight w_i by adding Δw_i, then repeat this process. This algorithm is given in Table 4.1. Because the error surface contains only a single global minimum, this algorithm will converge to a weight vector with minimum error, regardless of whether the training examples are linearly separable, given a sufficiently small learning rate η is used. If η is too large, the gradient descent search runs the risk of overstepping the minimum in the error surface rather than settling into it. For this reason, one common modification to the algorithm is to gradually reduce the value of η as the number of gradient descent steps grows.

Q.3 b) To calculate the output of a neuron Y in a neural network using

i) Unipolar sigmoid activation function and

ii) tan h activation function. (2 Marks)

Inputs: $x_1 = 1.0$, $x_2 = -2.0$, $x_3 = 3.0$
Weights: $w_1 = 0.2$, $w_2 = -0.5$, $w_3 = 0.7$
Bias: $b = w_0 = 0.4$

Solution:

		Net Input Calculation The net input to the neuron (often denoted as z or "net") is calculated as the weighted sum of the inputs plus the bias: **Step 1: Compute the Weighted Sum z**: $z = (0.2 \times 1.0) + (-0.5 \times -0.2) + (0.7 \times 3.0) + 0.4 = 3.7$ **Step 2: Apply Unipolar Sigmoid Activation Function:** $\sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-3.7}} = 0.97587$ **Step 3: Apply tah h Activation Function:** $\sigma(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} = \frac{e^{3.7}-e^{-3.7}}{e^{3.7}+e^{-3.7}} = 0.99877$ Result: Using, Sigmoid Activation Output $Y_{sigmoid}Y_{sigmoid}$: Approximately 0.97587 & using, Tanh Activation Output $Y_{tanh}Y_{tanh}$: Approximately 0.99877.